Lambdas and Functional Interfaces

9 min read · Updated 28 Jul 2026 · Topic 31 of 65

Why it matters

Lambdas changed how Java is written, and the questions about them are rarely about syntax. Interviewers ask what a lambda compiles to (not an anonymous class), what `this` refers to inside one (not the lambda), and why a lambda cannot capture a mutable local. Each answer reveals whether you understand the feature or merely use it.

The concept, simply

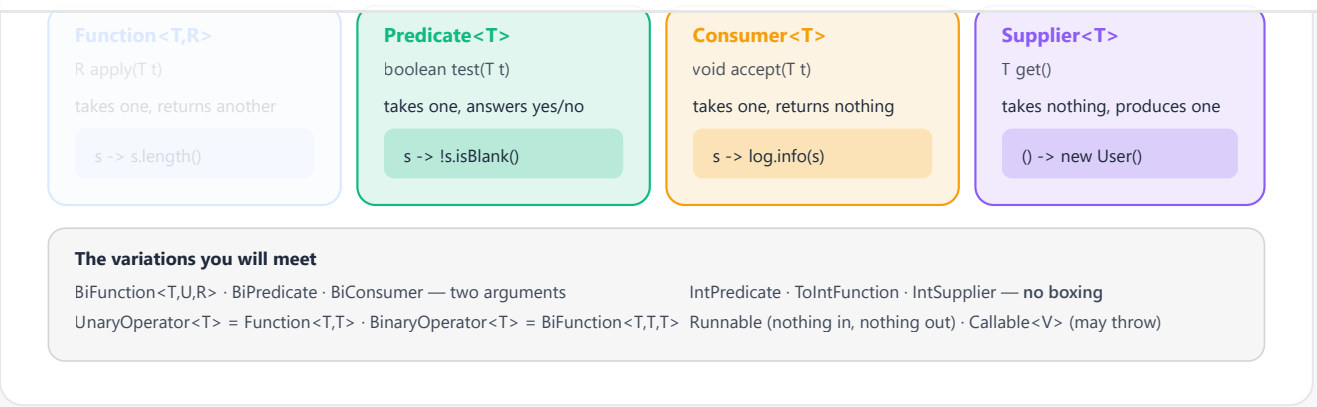
A **functional interface** is an interface with exactly one abstract method. A **lambda** is a compact way to supply an implementation of that one method.

JAVA Copy

```java
// Before Java 8 — an anonymous class, six lines of ceremony for one expre
Runnable r = new Runnable() {
    @Override public void run() { System.out.println("hi"); }
};

// Java 8+ — the same thing.
Runnable r = () -> System.out.println("hi");
```

The four shapes almost every lambda fits into come from `java.util.function`:



Learn these four and the rest of java.util.function is just variations on arity and primitive specialisation.

`@FunctionalInterface` is optional but worth applying: it makes the compiler verify that the interface really does have exactly one abstract method, so someone adding a second one gets a build error instead of silently breaking every lambda that targets it. Default and static methods do not count toward the total, which is why `Comparator` — with its dozen default methods — is still functional.

How it works

Effectively final capture. A lambda may use local variables from its enclosing scope only if they are `final` or *effectively final* — never reassigned after initialisation. This is not arbitrary: locals live on the stack and the stack frame may be gone by the time the lambda runs, so the value is *copied* into the lambda. Allowing reassignment would mean two copies that silently disagree. Instance and static fields have no such restriction, because they live on the heap and are genuinely shared.

`this` **means the enclosing instance.** A lambda is not a new object with its own identity, so `this` inside it refers to the class that contains it. In an anonymous class, `this` refers to the anonymous instance. That difference bites when converting old code.

Method references are lambdas whose body does nothing but call one existing method, in four forms:

Static	`Integer::parseInt`	`s -> Integer.parseInt(s)`
Instance of a particular object	`log::info`	`s -> log.info(s)`
Instance of an arbitrary object	`String::length`	`s -> s.length()`
Constructor	`ArrayList::new`	`() -> new ArrayList<>()`

Code examples

JAVA Copy

```java
// Syntax, from most explicit to most compact:
Comparator<String> c1 = (String a, String b) -> { return a.compareTo(b); }
Comparator<String> c2 = (a, b) -> a.compareTo(b);    // types inferred
Comparator<String> c3 = String::compareTo;           // method reference

Predicate<String> p = s -> !s.isBlank();             // one arg needs no pa
Supplier<List<String>> s = ArrayList::new;
Runnable r = () -> doWork();                         // zero args need them
```

The capture rule, and the workaround people reach for that is wrong:

JAVA Copy

```java
int count = 0;
List<String> items = List.of("a", "b");

// ✗ "local variables referenced from a lambda expression must be final
//    or effectively final"
// items.forEach(i -> count++);

// ✗ Technically compiles — the ARRAY reference is effectively final, and
//   people use this trick constantly. It is a workaround, not a fix: it i
//   not thread-safe, and it hides that you wanted a fold, not a mutation.
int[] counter = {0};
items.forEach(i -> counter[0]++);
```

```
long count = items.stream().filter(this::matches).count();
int total = items.stream().mapToInt(String::length).sum();
```

The `this` difference, which is a favourite question:

JAVA Copy

```
public class Widget {
    private String name = "widget";

    void withLambda() {
        Runnable r = () -> System.out.println(this.name);    // ✓ "widget"
        r.run();                                             // this == the
    }

    void withAnonymousClass() {
        Runnable r = new Runnable() {
            // private String name = "anon";
            @Override public void run() {
                // System.out.println(this.name);    → the ANONYMOUS instan
                System.out.println(Widget.this.name);  // must qualify to
            }
        };
        r.run();
    }
}
```

Method references and the one that surprises people:

JAVA Copy

```
list.forEach(System.out::println);            // instance of a particular
names.stream().map(String::toUpperCase);      // instance of an ARBITRARY
                                              // the stream element become
names.stream().map(String::valueOf);          // static

// ⚠ String::length works as Function<String,Integer> because the receiver
```

Under the hood

A lambda is not an anonymous class, and this is the answer that separates candidates. `javac` compiles the lambda body into a private synthetic method in the enclosing class and emits an `invokedynamic` instruction at the use site. At first execution, `LambdaMetafactory` spins up the implementing class and links the call site; every later execution goes straight to the linked target.

Three practical consequences:

- **No class file per lambda.** An anonymous class produces `Outer$1.class`; a lambda produces none. On a large codebase that is a real reduction in class count and load time.
- **Non-capturing lambdas are cached.** A lambda that captures nothing is instantiated once and reused, so it allocates nothing on repeated calls. A capturing lambda generally allocates per evaluation.
- **The strategy can change without recompiling.** Because the linkage is decided at run time, the JDK can improve lambda implementation and existing bytecode benefits — which was the actual motivation for using `invokedynamic` rather than generating classes at compile time.

Lambdas cannot throw checked exceptions unless the target interface declares them, and none of the standard ones do — Chapter 6's problem, and the reason `UncheckedIOException` exists.

Debugging is genuinely worse, and it is fair to say so: stack traces contain synthetic frames like `lambda$process$0`, and you cannot set a breakpoint on a lambda as naturally as on a method body. That is a real argument for extracting anything non-trivial into a named method — which also makes it testable.

Common mistakes

- **Trying to mutate a captured local.** It must be effectively final. Use a reduction rather than an array-of-one hack.
- **Expecting `this` to mean the lambda.** It means the enclosing instance — unlike an anonymous class.
- **Long lambda bodies.** Anything beyond a couple of lines belongs in a named method: readable, testable, and it appears properly in stack traces.
- **Side effects inside lambdas.** Mutating shared state from a lambda — especially in a stream — is where parallel bugs come from.
- **Omitting `@FunctionalInterface`.** Without it, someone can add a second abstract method and break every caller with no warning.
- **Expecting checked exceptions to work.** They do not; wrap them.
- **Assuming a lambda is an anonymous class.** Different mechanism, different `this`, no generated class file.

Interview lens

Q: "What is a functional interface?"

A: An interface with exactly one abstract method, which is what makes it a valid target for a lambda or method reference. Default and static methods do not count, which is why `Comparator` qualifies despite having many. `@FunctionalInterface` asks the compiler to enforce the rule so nobody accidentally adds a second abstract method.

Q: "Is a lambda just an anonymous inner class?"

A: No. The body compiles to a private synthetic method and the use site becomes an `invokedynamic` instruction linked at run time by `LambdaMetafactory`. So no extra class file is generated, non-capturing lambdas are cached and allocate nothing on reuse, and `this` refers to the enclosing instance rather than to a new object.

A: Because the value is copied into the lambda. Locals live on the stack frame, which may be gone by the time the lambda runs — a lambda can outlive the method that created it. If the variable could be reassigned there would be two copies that silently disagree. Fields are exempt, because they live on the heap and really are shared.

Q: "What does `this` mean inside a lambda?"

A: The enclosing instance — a lambda introduces no new scope for `this`. In an anonymous class it refers to the anonymous object, so you need `Outer.this` to reach the enclosing one. It is the most common surprise when converting anonymous classes to lambdas.

Q: "What kinds of method reference are there?"

A: Four. Static (`Integer::parseInt`), bound to a particular instance (`log::info`), unbound on an arbitrary instance where the stream element becomes the receiver (`String::length`), and constructor (`ArrayList::new`). The unbound form is the one that surprises people, because the receiver appears as the argument.

Q: "Can a lambda throw a checked exception?"

A: Only if the target functional interface declares it, and none of the standard ones do. So in practice you catch inside the lambda and wrap in an unchecked exception — the JDK's own `UncheckedIOException` exists for exactly this — or define your own throwing interface with an adapter.

Cheat-card

- Functional interface = **exactly one abstract method**; defaults/statics do not count.

- **No class file** per lambda; non-capturing lambdas are **cached**.
- Captured locals must be **effectively final** — the value is copied.
- `this` = **enclosing instance** (anonymous class: the anonymous object).
- Four shapes: `Function` · `Predicate` · `Consumer` · `Supplier`.
- Method refs: static · bound · unbound · constructor.
- No checked exceptions — wrap them.
- Long body? **Extract a named method** — readable, testable, better traces.

QUICK CHECK — no marks, just for you.

1. Is a lambda compiled into an anonymous inner class?

- ◯ Yes, but only when it captures variables
- ◯ No — lambdas are interpreted and never compiled
- ◯ Yes — the compiler generates Outer$1.class for each one
- ◯ No — the body becomes a private synthetic method and the use site becomes an invokedynamic instruction linked at run time by LambdaMetafactory

Check answer

2. Why must a local variable captured by a lambda be final or effectively final?

- ◯ Because the VALUE is copied into the lambda, and a lambda can outlive the stack frame — allowing reassignment would create two copies that silently disagree
- ◯ Because lambdas run on a different thread
- ◯ To prevent the lambda from being garbage collected

Because the compiler cannot infer the type otherwise

Check answer

3. What does 'this' refer to inside a lambda?

- ○ null
- ○ The enclosing instance — a lambda introduces no new scope for this, unlike an anonymous class
- ○ The functional interface being implemented
- ○ The lambda object itself

Check answer

‹ Previous
Designing Custom Exceptions

Next ›
Streams: Laziness and the Pipeline

LEARN MORE

ARTICLES & DOCS

- Lambda expressions — The Java Tutorials
  docs.oracle.com
- java.util.function — API docs
  docs.oracle.com
- Method references — The Java Tutorials
  docs.oracle.com

VIDEOS

Java Interview Kit



JAVA 8 Features Tutorial | Functional Interfaces | Lambda Expressions | Optional Class | Stream API

Dilip IT Academy



Java Lambda and Stream Full Course | Zero to Hero | Complete Course

Java Puzzle

Hand-picked references for this lesson.